

RETAILMART V3 • SQL

Aggregate Functions and GROUP BY

WhatsApp Announcement

AGGREGATE FUNCTIONS & GROUP BY

'How many orders per month? What is the average salary by role? Which store has the most revenue?' Today you learn to SUMMARIZE entire tables into business reports.

Today's Focus:

- COUNT, SUM, AVG, MIN, MAX — the big five aggregates
- GROUP BY — summarize by category
- HAVING — filter GROUPS (not rows)
- STRING_AGG — concatenate text in groups

This is where SQL becomes a reporting powerhouse!

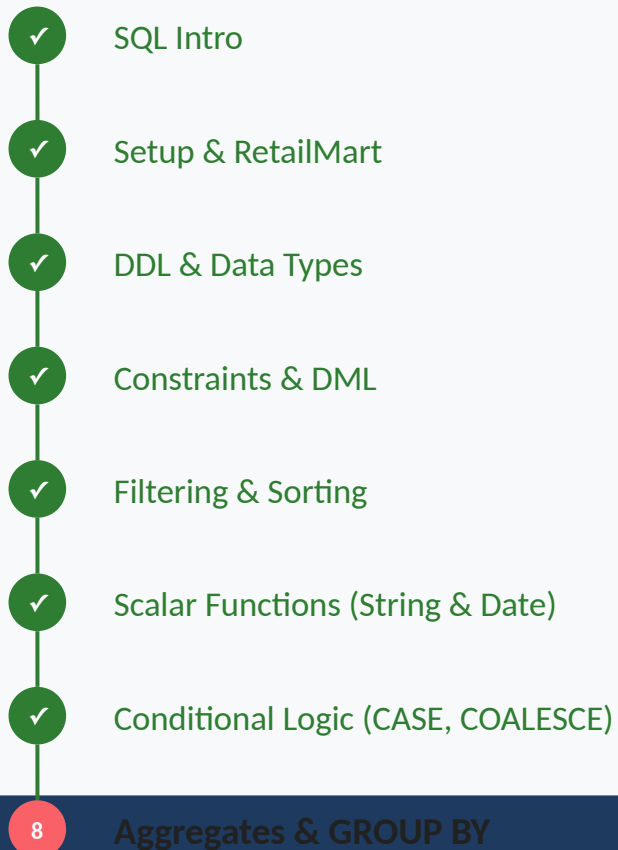
— Siraj Sir

#Day8 #GROUPBY #Aggregates #PostgreSQL

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Quick Recap — earlier sessions	5 mins	CASE WHEN, COALESCE, CAST from Conditional Logic, Scalar vs Aggregate functions
Part 1: Aggregate Functions	50 mins	COUNT — counting rows + practice, SUM & AVG — totals and averages + practice, MIN & MAX — finding extremes + practice, Combined hard & crazy problems (all aggregates)
Part 2: GROUP BY & HAVING	50 mins	GROUP BY — summarize by category + practice, HAVING — filter groups after aggregation + practice, SQL execution order, Combined hard & crazy problems
Part 3: STRING_AGG & Wrap-up	15 mins	STRING_AGG — concatenate text in groups, Practice problems
Quiz + Summary & Wrap-up	20 mins	Quick quiz (6 questions), Common mistakes gallery, Interview bank — aggregates, Syntax Reference Card, Homework + Joins Part 1 preview

YOUR SQL JOURNEY



Memory Check

SELECT, WHERE, AND/OR, LIKE, ORDER BY, LIMIT — finding data.

String + Date functions — transforming data (UPPER, TRIM, TO_CHAR, EXTRACT).

CASE WHEN, COALESCE, NULLIF, CAST — decision-making and NULL handling.

Today: We can FIND, TRANSFORM, and DECIDE. Now we SUMMARIZE — turn 150,000 order rows into one revenue number. Or split by category: revenue PER store, PER month, PER product.

The Principal's Impossible Request

The principal asks: 'How many students per class? What is the average score per subject? Which class has the highest topper? Which subjects have an average below 50?'

With 2,000 students and 8 subjects, you cannot count manually. But SQL can:

- COUNT gives totals per group
- AVG gives averages per subject
- MAX finds toppers per class
- GROUP BY splits by class or subject
- HAVING filters — only classes with avg < 50

From Story to SQL!

Students per class → `SELECT class, COUNT(*) GROUP BY class`

Average per subject → `SELECT subject, AVG(marks) GROUP BY subject`

Highest score → `SELECT class, MAX(marks) GROUP BY class`

Classes with avg < 50 → `HAVING AVG(marks) < 50`

One SQL query replaces hours of manual calculation. This is the POWER of aggregation.

Scalar vs Aggregate

earlier sessions functions (UPPER, LENGTH, CASE) work on ONE row and return ONE value — these are SCALAR functions.

Today's functions work on MANY rows and return ONE value — these are AGGREGATE functions. COUNT(*) takes 50,000 customer rows and returns a single number: 50000.

The Five Aggregate Functions

Technical:

COUNT(*) = total rows. COUNT(col) = non-NULL rows. COUNT(DISTINCT col) = unique values. SUM(col) = total of all values. AVG(col) = arithmetic mean (skips NULLs). MIN(col) = smallest value. MAX(col) = largest value.

Layman's:

COUNT = 'How many?' SUM = 'What is the total?' AVG = 'What is the average?' MIN = 'What is the smallest?' MAX = 'What is the largest?' They answer summary questions about your entire dataset.

SYNTAX: Three Flavors of COUNT

```
-- COUNT(*) – count ALL rows (including NULLs)
SELECT COUNT(*) FROM customers.customers;
    -- 50000 (every row counts)

-- COUNT(column) – count NON-NULL values only
SELECT COUNT(phone) FROM customers.customers;
    -- ~47500 (skips ~2500 NULL phones)

-- COUNT(DISTINCT col) – unique non-NULL values
SELECT COUNT(DISTINCT tier)
FROM customers.customers;
    -- 3 (Gold, Silver, Bronze)
```

PRO TIP

COUNT(*) vs COUNT(col): COUNT(*) counts ALL rows — even if every column is NULL. COUNT(phone) skips rows where phone IS NULL. The difference between COUNT(*) and COUNT(phone) tells you how many NULLs exist: COUNT(*) - COUNT(phone) = NULL count.

Easy: Basic Counts on RetailMart

EASY

SCENARIO: The CEO wants 3 quick numbers for the morning standup: total customers, total orders, and total products.

TASK: Write one SELECT using subqueries to show all 3 counts. Each subquery uses COUNT(*) on one table.

HINT: Think of each table's count as a scalar subquery (returns one number) — they all live side-by-side in a single SELECT.

Trap: each subquery must be wrapped in its own parentheses; missing one closes the entire SELECT prematurely.

Answer

 ANSWER

```
SELECT
  (SELECT COUNT(*)
   FROM customers.customers)
  AS total_customers,
  (SELECT COUNT(*)
   FROM sales.orders)
  AS total_orders,
  (SELECT COUNT(*)
   FROM products.products)
  AS total_products;
```

Medium: COUNT Variations — NULL Detection

MEDIUM

SCENARIO: The data team wants to understand NULL distribution in customers. How many total? How many have phones? How many NULLs? What percentage? How many unique email domains?

TASK: Use COUNT(*), COUNT(phone), subtraction for NULL count, ROUND for percentage, and COUNT(DISTINCT SPLIT_PART(email, '@', 2)) for unique domains. All in one query.

HINT: Two COUNT variants behave differently with NULLs — one counts every row, the other counts only non-NULL values. Trap: COUNT(column_name) is NOT the same as COUNT(*) — picking the wrong one silently changes the result without raising any error.

Answer



```
SELECT
  COUNT(*) AS total_customers,
  COUNT(phone) AS has_phone,
  COUNT(*) - COUNT(phone)
    AS missing_phone,
  ROUND(100.0 *
    (COUNT(*) - COUNT(phone))
    / COUNT(*), 2) AS null_pct,
  COUNT(DISTINCT
    SPLIT_PART(email, '@', 2))
    AS unique_domains,
  COUNT(DISTINCT tier) AS tier_count
FROM customers.customers;
```

SYNTAX: SUM and AVG

```
-- SUM: add up all values in a column
SELECT SUM(net_total) AS total_revenue
FROM sales.orders;
```

```
-- AVG: average of all values (skips NULLs!)
SELECT ROUND(AVG(net_total), 2)
       AS avg_order_value
FROM sales.orders;
```

```
-- Both on the same table:
SELECT
    SUM(salary) AS total_salary_cost,
    ROUND(AVG(salary), 0) AS avg_salary
FROM stores.employees;
```


AVG and NULLs — Silent Miscalculation

The Mistake:

AVG(salary) with 100 rows where 20 are NULL. AVG divides by 80 (non-NULL rows), NOT by 100. This gives a higher average than expected.

The Fix:

Use COALESCE if NULLs should count as 0: AVG(COALESCE(salary, 0)) divides by all 100 rows. Know which behavior you need BEFORE writing the query.

Easy: Revenue Totals

EASY

SCENARIO: Finance needs total revenue from ALL orders and total revenue from DELIVERED orders only.

TASK: Write two separate queries — one with SUM(net_total) on all orders, one with a WHERE filter for delivered orders. Include COUNT(*) in both.

HINT: Aggregates run AFTER filtering — think about query execution order. Trap: adding the filter in HAVING instead of WHERE forces the database to aggregate ALL rows first, then throw most away — wrong logically AND slow.

Answer

✓ ANSWER

```
-- All orders:
SELECT
    COUNT(*) AS total_orders,
    SUM(net_total) AS total_revenue,
    ROUND(AVG(net_total), 2) AS avg_order
FROM sales.orders;
```

```
-- Delivered only:
SELECT
    COUNT(*) AS delivered_orders,
    SUM(net_total) AS delivered_revenue,
    ROUND(AVG(net_total), 2)
    AS avg_delivered_order
FROM sales.orders
WHERE order_status = 'Delivered';
```

Medium: Employee Salary Analysis

MEDIUM

SCENARIO: HR wants a complete salary report: total salary cost, average salary (rounded), count of employees, and a comparison — total salary vs salary if NULLs were counted as 0.

TASK: Show total_salary (SUM), avg_salary (AVG + ROUND), avg_with_nulls (AVG(COALESCE)), and employee_count (COUNT). All from stores.employees.

HINT: AVG silently skips NULLs — which changes both the numerator AND the denominator. Trap: AVG(COALESCE(col, 0)) gives a DIFFERENT answer than AVG(col) when NULLs exist, because the zeros now count as real rows in the divisor.

Answer

 ANSWER

```
SELECT
  COUNT(*) AS employee_count,
  SUM(salary) AS total_salary,
  ROUND(AVG(salary), 0)
    AS avg_salary,
  ROUND(AVG(COALESCE(salary, 0)), 0)
    AS avg_with_nulls_as_zero,
  COUNT(salary) AS has_salary,
  COUNT(*) - COUNT(salary)
    AS missing_salary
FROM stores.employees;
```

SYNTAX: MIN and MAX

-- MIN & MAX on numbers

```
SELECT
    MIN(price) AS cheapest,
    MAX(price) AS most_expensive
FROM products.products;
```

-- MIN & MAX on dates (earliest/latest)

```
SELECT
    MIN(order_date) AS first_order,
    MAX(order_date) AS last_order
FROM sales.orders;
```

-- MIN & MAX on text (alphabetical)

```
SELECT
    MIN(first_name) AS alpha_first,
    MAX(first_name) AS alpha_last
FROM customers.customers;
```

PRO TIP

PRO TIP

MIN and MAX work on dates too! MIN(order_date) = earliest order. MAX(order_date) = latest order. They also work on text (alphabetical order). This makes them useful for finding date ranges and boundary values.

Easy: Finding Extremes

EASY

SCENARIO: The product team wants to know the cheapest and most expensive product, plus the earliest and latest order dates.

TASK: Query 1 — MIN/MAX price from products.products. Query 2 — MIN/MAX order_date from sales.orders (cast to DATE).

HINT: MIN and MAX work on more than just numbers — they sort dates and text too. Trap: displaying a timestamp with the time part can clutter reports — cast it to DATE to strip the time portion.

Answer

✓ ANSWER

-- Product price extremes:

```
SELECT
    MIN(price) AS cheapest,
    MAX(price) AS most_expensive,
    MAX(price) - MIN(price) AS price_range
FROM products.products;
```

-- Order date range:

```
SELECT
    MIN(order_date)::DATE AS first_order,
    MAX(order_date)::DATE AS last_order,
    MAX(order_date)::DATE -
        MIN(order_date)::DATE
        AS days_span
FROM sales.orders;
```

Medium: Product Margin Range Analysis

MEDIUM

SCENARIO: Finance wants margin analysis on products: cheapest, most expensive, price range, cost range, average margin, min margin, and max margin.

TASK: Use MIN, MAX, AVG on both price and cost_price. Calculate margin as price - cost_price. Use ROUND for averages. All in one query.

HINT: Aggregates accept expressions, not just plain columns — think about doing the math FIRST, then aggregating the result. Trap: do not aggregate one side and not the other — $\text{MIN}(\text{price}) - \text{MAX}(\text{cost_price})$ gives a totally different answer than $\text{MIN}(\text{price} - \text{cost_price})$.

Answer



ANSWER

```
SELECT
  COUNT(*) AS total_products,
  MIN(price) AS min_price,
  MAX(price) AS max_price,
  ROUND(AVG(price), 0) AS avg_price,
  MIN(price - cost_price)
    AS min_margin,
  MAX(price - cost_price)
    AS max_margin,
  ROUND(AVG(price - cost_price), 0)
    AS avg_margin
FROM products.products;
```

The Five Aggregates — Every Analyst Uses These Daily

COUNT = 'How many?' — report totals, data quality checks

SUM = 'What is the total?' — revenue, costs, quantities

AVG = 'What is the average?' — order value, response time, salary

MIN = 'What is the smallest/earliest?' — first order, cheapest product

MAX = 'What is the largest/latest?' — biggest order, latest login

Master these five and you can answer 80% of all business questions.

Hard: Complete Order Statistics

HARD

SCENARIO: Finance needs a comprehensive order report: total orders, total revenue, average order value, smallest and largest order, unique customer count, and date range — all in one query.

TASK: Combine all five aggregates on sales.orders. Use ROUND for the average. Show date range with MIN/MAX.

HINT: Multiple aggregates can sit in one SELECT — the database computes each in a single pass over the data. Trap: if you mix aggregates with a plain column (no GROUP BY), PostgreSQL throws 'must appear in GROUP BY or aggregate function'.

Answer

✓ ANSWER

```
SELECT
  COUNT(*) AS total_orders,
  SUM(net_total) AS total_revenue,
  ROUND(AVG(net_total), 2) AS avg_order,
  MIN(net_total) AS smallest_order,
  MAX(net_total) AS largest_order,
  COUNT(DISTINCT cust_id)
    AS unique_customers,
  MIN(order_date)::DATE AS first_order,
  MAX(order_date)::DATE AS last_order
FROM sales.orders;
```

Crazy: Marketing Ad Spend Dashboard

CRAZY

SCENARIO: The Marketing Director needs a one-row platform-spend dashboard from `marketing.ads_spend`. Show: total spend, count of spend records, distinct platforms used, average daily spend per platform, biggest single spend, and a 'spend on Google vs Meta' comparison using CASE inside SUM.

TASK: Combine `COUNT(*)`, `COUNT(DISTINCT platform)`, `SUM`, `AVG`, `MAX`, and CASE-inside-SUM for platform-specific totals. No `GROUP BY` (single-row dashboard).

HINT: To get platform-specific totals on a single row, put a CASE inside SUM — only matching rows contribute. Trap: `COUNT(DISTINCT platform)` ignores NULLs; if some rows have NULL platform you'll undercount — verify with `COUNT(DISTINCT COALESCE(platform, 'Unknown'))`.

Answer

✓ ANSWER

```
SELECT
  COUNT(*) AS total_records,
  COUNT(DISTINCT platform)
    AS unique_platforms,
  ROUND(SUM(amount), 0) AS total_spend,
  ROUND(AVG(amount), 0) AS avg_spend,
  ROUND(MAX(amount), 0)
    AS biggest_single_spend,
  ROUND(SUM(CASE WHEN platform = 'Google'
    THEN amount END), 0)
    AS google_total,
  ROUND(SUM(CASE WHEN platform = 'Meta'
    THEN amount END), 0)
    AS meta_total,
  ROUND(SUM(CASE WHEN platform = 'YouTube'
    THEN amount END), 0)
    AS youtube_total
FROM marketing.ads_spend;
```


The Report Card Per Subject

The principal does not want the school's OVERALL average. She wants the average PER SUBJECT: Math 72, Science 68, English 81.

GROUP BY is like sorting papers into piles by subject, then calculating stats for each pile separately. You get ONE result row PER group.

GROUP BY — Make Separate Piles

Technical:

GROUP BY divides rows into groups based on one or more columns, then applies aggregate functions to each group independently. Every non-aggregated column in SELECT must appear in GROUP BY.

Layman's:

GROUP BY = 'make separate piles'. If you GROUP BY city, all Mumbai rows go in one pile, all Delhi rows in another. COUNT, SUM, AVG work on each pile separately. You get one result row PER pile.

SYNTAX: GROUP BY

-- Basic GROUP BY

```
SELECT column, AGG_FUNC(other_column)
FROM table
GROUP BY column;
```

-- THE GOLDEN RULE:

-- Every column in SELECT that is NOT inside
-- an aggregate function MUST be in GROUP BY!

-- Multiple columns

```
SELECT col1, col2, COUNT(*)
FROM table
GROUP BY col1, col2;
```

GROUP BY with CASE — Custom Groups (1/2)

```
-- Group orders by custom value tier
SELECT
  CASE
    WHEN net_total > 25000 THEN 'High Value'
    WHEN net_total > 10000 THEN 'Medium'
    WHEN net_total > 5000 THEN 'Standard'
    ELSE 'Small'
  END AS order_tier,
  COUNT(*) AS order_count,
  ROUND(SUM(net_total), 0) AS total_revenue
FROM sales.orders
GROUP BY
  CASE
    WHEN net_total > 25000 THEN 'High Value'
    WHEN net_total > 10000 THEN 'Medium'
    WHEN net_total > 5000 THEN 'Standard'
    ELSE 'Small'
  END
ORDER BY total_revenue DESC;
```

GROUP BY with CASE — Custom Groups (2/2)

```
-- Conditional Logic CASE + Aggregates & Grouping GROUP BY = powerful!
```

The Golden Rule Violation

The Mistake:

`SELECT store_id, order_status, COUNT(*) FROM sales.orders GROUP BY store_id` — ERROR! `order_status` is in `SELECT` but NOT in `GROUP BY` and NOT inside an aggregate.

The Fix:

Either add `order_status` to `GROUP BY` (one row per store+status combo), or remove it from `SELECT`, or wrap it in an aggregate like `STRING_AGG(DISTINCT order_status, ', ')`.

Easy: Orders Per Status

EASY

SCENARIO: The operations manager asks: how many orders are in each status? She needs a simple status breakdown sorted by count.

TASK: Group orders by order_status and count each group. Sort by count descending.

HINT: Counting per category means grouping by that category — one row per distinct value. Trap: the column you're grouping by MUST appear unaggregated in SELECT; otherwise GROUP BY does nothing useful.

Answer



ANSWER

```
SELECT
    order_status,
    COUNT(*) AS order_count
FROM sales.orders
GROUP BY order_status
ORDER BY order_count DESC;
```


Medium: Monthly Revenue Report

MEDIUM

SCENARIO: Finance wants a monthly revenue trend: month, order count, total revenue, and average order value. This is the #1 analytics pattern in SQL.

TASK: Use `DATE_TRUNC('month', order_date)` for grouping. Show month label with `TO_CHAR`. Include `COUNT`, `SUM`, and `ROUND(AVG)`. Sort by month.

HINT: Time-series reporting needs you to `BUCKET` timestamps to a common unit (month, week, day) before grouping.

Trap: whatever transformation you use in `SELECT (DATE_TRUNC, TO_CHAR)` must appear identically in `GROUP BY` — aliases are not allowed there.

Answer

 ANSWER

```
SELECT
    DATE_TRUNC('month', order_date)::DATE
        AS month,
    TO_CHAR(order_date, 'Mon YYYY')
        AS month_label,
    COUNT(*) AS orders,
    ROUND(SUM(net_total), 0) AS revenue,
    ROUND(AVG(net_total), 0) AS avg_order
FROM sales.orders
GROUP BY
    DATE_TRUNC('month', order_date),
    TO_CHAR(order_date, 'Mon YYYY')
ORDER BY month;
```

PRO TIP

`DATE_TRUNC('month', date) + GROUP BY` is the most important analytics pattern in SQL. It creates monthly/quarterly/yearly reports. Combined with `SUM`, `COUNT`, and `AVG`, it builds the revenue trends, order trends, and KPI dashboards that every business uses.

HAVING vs WHERE

WHERE = bouncer at the door. Checks EACH ROW before it enters the group. Runs BEFORE GROUP BY.

HAVING = judge after the contest. Evaluates EACH GROUP and eliminates low scorers. Runs AFTER GROUP BY.

WHERE says: 'only let delivered orders in'. HAVING says: 'only show stores with revenue above 5 lakh'.

SYNTAX: HAVING

-- Basic HAVING – filter groups

```
SELECT column, COUNT(*) AS cnt
FROM table
GROUP BY column
HAVING COUNT(*) > 100;
```

-- WHERE + HAVING together

```
SELECT column, SUM(amount) AS total
FROM table
WHERE row_condition      -- filters rows FIRST
GROUP BY column
HAVING SUM(amount) > 500000; -- then groups
```

-- Multiple HAVING conditions

```
SELECT col, AVG(val) AS avg_val
FROM table
GROUP BY col
HAVING AVG(val) > 100 AND COUNT(*) >= 5;
```

PRO TIP

Put as much filtering in WHERE as possible — it reduces the number of rows BEFORE GROUP BY runs. HAVING should only be used for conditions on aggregate results (COUNT, SUM, AVG). Using HAVING for row-level filters works but is slower because all rows are grouped first, then thrown away.

Easy: High-Volume Stores

EASY

SCENARIO: The VP of Sales wants to see only stores with 500+ orders. Small stores are not relevant for the quarterly review.

TASK: GROUP BY store_id with COUNT and SUM(net_total). Use HAVING COUNT(*) >= 500 to filter. Order by revenue descending.

HINT: WHERE filters individual rows; HAVING filters groups AFTER aggregation. Trap: putting the aggregate condition in WHERE throws an error — WHERE has no access to aggregate results yet.

Answer

 ANSWER

```
SELECT
    store_id,
    COUNT(*) AS order_count,
    ROUND(SUM(net_total), 0) AS revenue
FROM sales.orders
GROUP BY store_id
HAVING COUNT(*) >= 500
ORDER BY revenue DESC;
```


Medium: Salary Statistics Per Role

MEDIUM

SCENARIO: HR wants salary stats per role: headcount, average salary, min, max. Only include roles with 5+ employees.

TASK: GROUP BY role with COUNT, ROUND(AVG), MIN, MAX on salary. Use HAVING to filter roles with fewer than 5 people.

HINT: Whenever you filter using a SUM, COUNT, or AVG result, the clause that handles it is HAVING — not WHERE. Trap: HAVING can use aggregates but cannot reference SELECT aliases — repeat the full aggregate expression.

Answer

 ANSWER

```
SELECT
    role,
    COUNT(*) AS emp_count,
    ROUND(AVG(salary), 0) AS avg_salary,
    MIN(salary) AS min_salary,
    MAX(salary) AS max_salary
FROM stores.employees
GROUP BY role
HAVING COUNT(*) >= 5
ORDER BY avg_salary DESC;
```

The Order SQL Actually Runs

You WRITE: SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT

SQL EXECUTES in this order:

1. FROM — pick the table
2. WHERE — filter individual rows
3. GROUP BY — make piles
4. HAVING — filter piles
5. SELECT — pick columns & compute expressions
6. DISTINCT — remove duplicates
7. ORDER BY — sort results
8. LIMIT — cap the output

This is why WHERE cannot use aliases (SELECT hasn't run yet) and WHERE cannot use COUNT (GROUP BY hasn't run yet).

Hard: Monthly Revenue — WHERE + GROUP BY + HAVING

HARD

SCENARIO: Finance wants monthly revenue for delivered orders. Only show months with 1000+ orders. Add a performance label using CASE.

TASK: Use DATE_TRUNC for monthly grouping. WHERE filters delivered orders. GROUP BY creates monthly piles. HAVING drops small months. CASE labels performance.

HINT: Three filtering moments — pre-aggregation (WHERE), grouping (GROUP BY), post-aggregation (HAVING). Trap: row-level conditions belong in WHERE for performance — putting them in HAVING forces the aggregation to process rows that will be thrown away.

Answer

✓ ANSWER

```
SELECT
  DATE_TRUNC('month', order_date)::DATE
  AS month,
  COUNT(*) AS total_orders,
  ROUND(SUM(net_total), 0) AS revenue,
  ROUND(AVG(net_total), 0) AS avg_order,
  CASE
    WHEN SUM(net_total) > 1000000
      THEN 'High'
    WHEN SUM(net_total) > 500000
      THEN 'Medium'
    ELSE 'Low'
  END AS performance
FROM sales.orders
WHERE order_status = 'Delivered'
GROUP BY DATE_TRUNC('month', order_date)
HAVING COUNT(*) >= 1000
ORDER BY month;
```

Crazy: Finance Expense Category Dashboard

CRAZY

SCENARIO: The CFO wants a monthly expense report by category. From `finance.expenses` joined to `dim_expense_category`, per category show: total spend, transaction count, avg per transaction, largest single expense, and YoY growth flag. Only categories with 20+ expenses this year.

TASK: JOIN `finance.expenses` to `core.dim_expense_category`. GROUP BY `category_name`. Aggregate the spend metrics. HAVING for the 20+ threshold. Add CASE-based YoY columns using `EXTRACT(YEAR)`.

HINT: Three filtering moments — pre-aggregation (WHERE), grouping (GROUP BY), post-aggregation (HAVING). Trap: HAVING runs AFTER GROUP BY so it can use aggregates; WHERE runs BEFORE so it can't reference SUM/COUNT — putting an aggregate condition in WHERE throws an error.

Answer



ANSWER

```
SELECT
    cat.category_name,
    COUNT(*) AS transactions,
    ROUND(SUM(e.amount), 0) AS total_spend,
    ROUND(AVG(e.amount), 0)
        AS avg_per_txn,
    ROUND(MAX(e.amount), 0)
        AS biggest_expense,
    ROUND(SUM(CASE WHEN EXTRACT(YEAR FROM
        e.expense_date) = 2024
        THEN e.amount END), 0)
        AS spend_2024,
    ROUND(SUM(CASE WHEN EXTRACT(YEAR FROM
        e.expense_date) = 2025
        THEN e.amount END), 0)
        AS spend_2025
FROM finance.expenses e
JOIN core.dim_expense_category cat
    ON e.exp_cat_id = cat.exp_cat_id
GROUP BY cat.category_name
HAVING COUNT(*) >= 20
ORDER BY total_spend DESC LIMIT 15;
```

The Birthday Invitation List

Mom asks: 'Which friends are coming?' Instead of listing one friend per row, you say: 'Aman, Priya, Rahul, Sneha.' That is `STRING_AGG` — combining multiple rows into one comma-separated string per group.

DEFINITION

STRING_AGG

Technical:

STRING_AGG(expression, delimiter) concatenates non-NULL string values from a group into one string, separated by the delimiter. Supports ORDER BY inside the function and DISTINCT for unique values.

Layman's:

STRING_AGG takes a column of separate values and glues them together with a separator. Like putting names on ONE line with commas between them.

SYNTAX: STRING_AGG (1/2)

-- Basic: combine values with separator

```
SELECT STRING_AGG(role, ', ')  
FROM stores.employees;
```

-- With DISTINCT (unique only)

```
SELECT STRING_AGG(DISTINCT role, ', '  
    ORDER BY role)  
FROM stores.employees;
```

-- With GROUP BY – per-group concatenation

```
SELECT store_id,  
    COUNT(*) AS headcount,  
    STRING_AGG(first_name, ', '  
        ORDER BY first_name)  
    AS team_members  
FROM stores.employees  
GROUP BY store_id;
```

-- Non-text columns must be cast first

```
SELECT STRING_AGG(store_id::TEXT, ', ')
```

SYNTAX: STRING_AGG (2/2)

```
FROM stores.stores;
```

PRO TIP

STRING_AGG is incredibly useful for reports where stakeholders want compact summaries. Instead of 15 rows showing 'Store 1: Cashier, Store 1: Manager, Store 1: Associate...', they get one row: 'Store 1: Cashier | Manager | Associate'. Add DISTINCT to avoid repeats.

Easy: Roles Per Store

EASY

SCENARIO: The HR dashboard wants a compact view: for each store, show headcount and all distinct roles as one comma-separated string.

TASK: GROUP BY store_id. Use COUNT for headcount, STRING_AGG(DISTINCT role, ' | ' ORDER BY role) for the role list. Only stores with store_id NOT NULL. Show top 10 by headcount.

HINT: STRING_AGG can take optional modifiers — one to dedupe within the group, another to control output order. Trap: ORDER BY inside STRING_AGG is NOT the same as the outer ORDER BY — they sort different things (output rows vs concatenated values).

Answer

✓ ANSWER

```
SELECT
    store_id,
    COUNT(*) AS headcount,
    STRING_AGG(DISTINCT role, ' | '
        ORDER BY role) AS roles
FROM stores.employees
WHERE store_id IS NOT NULL
GROUP BY store_id
ORDER BY headcount DESC
LIMIT 10;
```

Medium: Department Summary with Role List

MEDIUM

SCENARIO: Build a department report: headcount, avg salary, salary range, and a compact role list. Only departments with 10+ people.

TASK: GROUP BY dept_id. Combine COUNT, AVG, MIN, MAX on salary with STRING_AGG for roles. HAVING count >= 10. Order by avg salary DESC.

HINT: You can mix STRING_AGG with the numeric aggregates in a single GROUP BY — they all consume the same grouped rows. Trap: a separator is REQUIRED in STRING_AGG — if you forget the second argument, you get a compile error.

Answer

 ANSWER

```
SELECT
    dept_id,
    COUNT(*) AS headcount,
    ROUND(AVG(salary), 0) AS avg_salary,
    MAX(salary) - MIN(salary)
    AS salary_range,
    STRING_AGG(DISTINCT role, ', '
    ORDER BY role) AS roles
FROM stores.employees
GROUP BY dept_id
HAVING COUNT(*) >= 10
ORDER BY avg_salary DESC;
```


Quick Fire Round!

Run each query on your RetailMart database. Modify the GROUP BY columns. Change HAVING thresholds. Add new aggregates. Understanding comes from experimentation.

Challenge 1: Customer Tier Distribution

```
-- How many customers per tier?
SELECT tier,
       COUNT(*) AS customer_count
FROM customers.customers
GROUP BY tier
ORDER BY customer_count DESC;

-- YOUR TURN: Add AVG(EXTRACT(YEAR FROM
-- AGE(registration_date))) as avg_tenure
```

Challenge 2: Quarterly Revenue

```
-- Revenue by quarter
SELECT
  EXTRACT(YEAR FROM order_date) AS yr,
  'Q' || EXTRACT(QUARTER FROM order_date)
    AS quarter,
  COUNT(*) AS orders,
  ROUND(SUM(net_total), 0) AS revenue
FROM sales.orders
GROUP BY yr, quarter
ORDER BY yr, quarter;

-- YOUR TURN: Add HAVING to show only
-- quarters with revenue > 5 lakh
```

Challenge 3: Employee Salary Report

```
-- Salary stats per department
SELECT dept_id,
       COUNT(*) AS headcount,
       ROUND(AVG(salary), 0) AS avg_sal,
       MAX(salary) - MIN(salary) AS salary_range,
       STRING_AGG(DISTINCT role, ', '
                  ORDER BY role) AS roles
FROM stores.employees
GROUP BY dept_id
HAVING COUNT(*) >= 10
ORDER BY avg_sal DESC;
```

Challenge 4: Order Value Distribution

```
-- Group orders by value category
SELECT
  CASE
    WHEN net_total > 25000 THEN 'High'
    WHEN net_total > 10000 THEN 'Medium'
    WHEN net_total > 5000 THEN 'Standard'
    ELSE 'Small'
  END AS order_tier,
  COUNT(*) AS order_count,
  ROUND(AVG(net_total), 0) AS avg_value
FROM sales.orders
GROUP BY order_tier
ORDER BY avg_value DESC;

-- YOUR TURN: Add SUM(net_total) and
-- percentage of total orders
```

Quiz 1: What does this return?

EASY

ANSWER: `COUNT(*)` = 50000 (all rows). `COUNT(phone)` $\approx$ 47500 (skips NULLs). The difference tells you ~2500 customers have NULL phone numbers.

Question

```
SELECT COUNT(*), COUNT(phone)
FROM customers.customers;
```



ANSWER

Quiz 2: Why does this crash?

MEDIUM

ANSWER: WHERE cannot use aggregate functions — GROUP BY hasn't run yet at the WHERE stage. Fix: move the condition to HAVING COUNT(*) > 100.

Question



ANSWER

```
SELECT store_id, COUNT(*)  
FROM sales.orders  
WHERE COUNT(*) > 100  
GROUP BY store_id;
```

Quiz 3: What is wrong here?

MEDIUM

ANSWER: `order_status` is in `SELECT` but NOT in `GROUP BY` and NOT inside an aggregate. The Golden Rule: every non-aggregated column in `SELECT` must be in `GROUP BY`. Fix: add `order_status` to `GROUP BY`, or remove it, or wrap in `STRING_AGG`.

Question



ANSWER

```
SELECT store_id, order_status, COUNT(*)  
FROM sales.orders  
GROUP BY store_id;
```

Quiz 4: WHERE or HAVING?

HARD

Where do these conditions go?

- a) 'Only count delivered orders' → WHERE order_status = 'Delivered' (row-level filter, before grouping)
- b) 'Only show stores with 500+ orders' → HAVING COUNT(*) >= 500 (group-level filter, after grouping)
- c) 'Only orders from 2025' → WHERE order_date >= '2025-01-01' (row-level filter)
- d) 'Only months with revenue > 10 lakh' → HAVING SUM(net_total) > 1000000 (aggregate filter)

Quiz 5: Predict the Result

CRAZY

ANSWER: total = 150000, stores = 200 (unique store_ids), revenue_per_store = total revenue divided by 200. NULLIF prevents divide-by-zero. COUNT(DISTINCT) counts unique values only. ROUND(..., 0) gives whole number.

Question

 ANSWER

```
SELECT
  COUNT(*) AS total,
  COUNT(DISTINCT store_id) AS stores,
  ROUND(SUM(net_total) /
    NULLIF(COUNT(DISTINCT store_id),
      0), 0) AS revenue_per_store
FROM sales.orders;
```

Quiz 6: What is the output difference?

HARD

ANSWER: Both return the SAME number! Query A filters rows first (WHERE). Query B counts matching rows with CASE (conditional aggregation). Same result, different technique. Query B is useful when you need multiple counts in one query.

Question

 ANSWER

```
-- Query A:  
SELECT COUNT(*) FROM sales.orders  
WHERE order_status = 'Delivered';
```

```
-- Query B:  
SELECT COUNT(CASE WHEN  
    order_status = 'Delivered'  
    THEN 1 END) FROM sales.orders;
```


WHERE vs HAVING — Wrong Filter Location

The Mistake:

WHERE COUNT(*) > 5 — crashes because COUNT doesn't exist at the WHERE stage. Another mistake: putting order_status = 'Delivered' in HAVING — works but slower because rows are grouped then thrown away.

The Fix:

Row-level filters → WHERE (before GROUP BY). Group-level filters → HAVING (after GROUP BY). Put as much in WHERE as possible for performance.

Integer Division in Percentages

The Mistake:

`COUNT(resolved_date) / COUNT(*) * 100` — all percentages show 0! `COUNT` returns `BIGINT`. `BIGINT / BIGINT` truncates:
 $850/1000 = 0$.

The Fix:

Cast numerator to `NUMERIC`: `COUNT(resolved_date)::NUMERIC / COUNT(*) * 100`. Also add `NULLIF(COUNT(*), 0)` to prevent divide-by-zero.

SELECT Alias in HAVING

The Mistake:

`SELECT COUNT(*) AS order_count ... HAVING order_count > 100` — error in standard SQL! HAVING runs before SELECT, so it doesn't know the alias.

The Fix:

Repeat the full expression: `HAVING COUNT(*) > 100`. PostgreSQL actually allows aliases in HAVING (non-standard), but repeating is safer and more portable.

WHERE vs HAVING

Q: 'What is the difference between WHERE and HAVING?'

A: WHERE filters individual ROWS before GROUP BY runs — it cannot use aggregate functions. HAVING filters GROUPS after GROUP BY — it is designed for aggregate conditions. Example: WHERE salary > 50000 filters employees. HAVING AVG(salary) > 50000 filters groups.

How AVG Handles NULLs

Q: 'How does AVG handle NULL values?'

A: AVG ignores NULLs. It equals $\text{SUM}(\text{col}) / \text{COUNT}(\text{col})$ — both skip NULLs. So 5 values plus 2 NULLs divides by 5, not 7.
If you want NULLs as 0: $\text{AVG}(\text{COALESCE}(\text{col}, 0))$. If ALL values are NULL, AVG returns NULL.

SQL Execution Order

Q: 'What is the SQL execution order?'

A: FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. This is why: WHERE can't use aliases (SELECT hasn't run), WHERE can't filter aggregates (GROUP BY hasn't run), ORDER BY CAN use aliases (runs after SELECT).

COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col)

Q: 'What is the difference between the three forms of COUNT?'

A: COUNT(*) counts ALL rows including NULLs. COUNT(col) counts rows where col is NOT NULL. COUNT(DISTINCT col) counts unique non-NULL values. Example: 100 rows with 20 NULL phones and 10 duplicate phones → COUNT(*) = 100, COUNT(phone) = 80, COUNT(DISTINCT phone) = 70.

GROUP BY with Multiple Columns

Q: 'What happens when you GROUP BY two columns?'

A: You get one row per unique COMBINATION. GROUP BY store_id, order_status with 200 stores $\times$ 7 statuses = up to 1,400 groups (fewer if some combos don't exist). Each group gets its own COUNT, SUM, AVG etc.

Can you GROUP BY a CASE expression?

Q: 'Can you use CASE WHEN in GROUP BY?'

A: Yes! GROUP BY CASE WHEN net_total > 10000 THEN 'High' ELSE 'Low' END creates custom groups based on computed categories. The full CASE expression must be repeated in GROUP BY (you cannot use the alias). This is how analysts create custom segmentation reports.

SUM vs COUNT for Totals

Q: 'When do you use SUM vs COUNT?'

A: COUNT tells you HOW MANY rows (count of orders). SUM tells you the TOTAL VALUE (total revenue). COUNT(CASE WHEN ...) counts matching rows. SUM(CASE WHEN ... THEN amount) sums matching amounts. Common mistake: using COUNT when you need SUM for monetary totals.

Real Interview: Monthly Growth Report

Q: 'Write a query showing monthly revenue and month-over-month order count.'

A: `SELECT DATE_TRUNC('month', order_date)::DATE AS month, COUNT(*) AS orders, ROUND(SUM(net_total), 0) AS revenue FROM sales.orders GROUP BY month ORDER BY month.` This tests GROUP BY + DATE_TRUNC + multiple aggregates — the #1 analytics pattern. Adding growth % requires window functions (Window Functions Part 3).

KEY TAKEAWAYS

AGGREGATE FUNCTIONS:

1. COUNT(*) = all rows. COUNT(col) = non-NULL. COUNT(DISTINCT) = unique.
2. SUM = total. AVG = mean (skips NULLs). MIN/MAX = extremes.
3. All aggregates ignore NULL (except COUNT(*)).

GROUP BY & HAVING:

4. GROUP BY creates 'piles' — one result row per group.
5. Every non-aggregated column in SELECT MUST be in GROUP BY.
6. WHERE filters rows BEFORE grouping. HAVING filters groups AFTER.
7. Execution: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

ADVANCED:

8. STRING_AGG(col, ' ', ') = concatenate values per group.
9. CASE inside aggregates = conditional counting/summing.
10. DATE_TRUNC + GROUP BY = monthly/quarterly reports.

Aggregate Functions and GROUP BY

-- COUNT

```
COUNT(*)           -- all rows
COUNT(column)     -- non-NULL values
COUNT(DISTINCT col) -- unique non-NULL
```

-- SUM / AVG

```
SUM(column)        -- total (numeric)
AVG(column)         -- mean (skips NULLs)
ROUND(AVG(col), 2) -- rounded average
```

-- MIN / MAX

```
MIN(column) -- smallest (num/date/text)
MAX(column) -- largest (num/date/text)
```

Aggregate Functions and GROUP BY

-- GROUP BY

```
SELECT col, AGG(x) FROM t GROUP BY col
-- non-agg cols MUST be in GROUP BY
DATE_TRUNC('month', date) for monthly
```

-- HAVING

```
GROUP BY col HAVING AGG(x) > value
-- filters GROUPS (after GROUP BY)
-- WHERE = rows, HAVING = groups
```

-- STRING_AGG

```
STRING_AGG(col, ', ' ORDER BY col)
STRING_AGG(DISTINCT col, ', ')
-- cast non-text: id::TEXT
```

CHALLENGE

Aggregates & Grouping Take-Home Challenge (1/2)

Build these queries on RetailMart V3:

Aggregates & Grouping Take-Home Challenge (2/2)

- 1.: Count total products, average price (ROUND), min and max price from products.products.
- 2.: Count orders per store_id. Only show stores with 500+ orders (HAVING). Sort by count DESC.
- 3.: Monthly revenue report: month, order count, total revenue, avg order value. Use DATE_TRUNC. HAVING revenue > 5 lakh.
- 4.: Salary statistics per role: headcount, avg/min/max salary, STRING_AGG of distinct dept_ids. HAVING count >= 5.
- 5.: Ticket category dashboard: count, resolved count, resolution rate %, priority breakdown (CASE inside COUNT). HAVING count >= 100.
- 6.: Build a complete status summary: per order_status show count, revenue, avg value, percentage of total orders, and date range.

Push your SQL file to GitHub!

<https://github.com/starlordali4444/postgresql>

WHAT YOU ACHIEVED TODAY

You mastered ALL aggregate functions: COUNT, SUM, AVG, MIN, MAX, STRING_AGG. You understand GROUP BY (the most important clause in analytics SQL) and HAVING (the group filter).

These are the building blocks of every business dashboard, every KPI report, every data analysis task.

Tomorrow: JOINS Part 1! You will learn to CONNECT tables — combining customer data with their orders, products with categories. That is when SQL becomes truly powerful.

Coming Tomorrow

Right now, orders and customers live in SEPARATE tables. To answer 'Which customer placed the most orders?' you need to CONNECT them. JOINS link tables using shared keys (customer_id, store_id, product_id).

INNER JOIN = only matching rows. LEFT JOIN = all from left + matches from right. RIGHT JOIN = all from right + matches from left. JOINS are where SQL becomes a real query LANGUAGE.